

Fairer Vergleich - Alle Modelle auf Breast Cancer

Datensatz: Breast Cancer Wisconsin (569 Samples, 2 Klassen: benign/malignant)

Features: 2 (nach PCA) → 2 Qubits, konsistent mit Iris-Experimenten

Split: test_size=0.3, random_state=42, stratify=y

Scaler: MinMaxScaler [0, 2 π]

F1: average='binary' (2 Klassen)

Modell	Typ
Quantum Kernel SVM	Quantenmodell
VQC (RealAmplitudes)	Quantenmodell
Klassischer SVM (RBF)	Klassisch
MLP (64, 32)	Klassisch

0. Imports

```
In [1]: import sys, os
        sys.path.append(os.path.abspath("."))

        import numpy as np
        import pandas as pd
        import time
        import warnings
        warnings.filterwarnings("ignore")

        from sklearn.datasets import load_breast_cancer
        from sklearn.model_selection import train_test_split
        from sklearn.preprocessing import MinMaxScaler
        from sklearn.decomposition import PCA
        from sklearn.metrics import accuracy_score, f1_score, classification_report
        from sklearn.svm import SVC
        from sklearn.neural_network import MLPClassifier

        from qiskit.circuit.library import zz_feature_map, real_amplitudes
        from qiskit_machine_learning.kernels import FidelityQuantumKernel
        from qiskit_machine_learning.algorithms import VQC
        from qiskit.primitives import StatevectorSampler
        from qiskit_machine_learning.optimizers import SPSA

        from utils import save_result

        print("Imports OK")
```

Imports OK

1. Datensatz & Preprocessing

Breast Cancer hat 30 Features - für 2 Qubits wird PCA auf 2 Komponenten reduziert. PCA wird auf Trainingsdaten gefittet, Testdaten werden nur transformiert (kein Data Leakage).

```
In [2]: bc = load_breast_cancer()
X = bc.data
y = bc.target # 0 = malignant, 1 = benign

# Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# PCA auf 2 Komponenten (fit nur auf Training)
pca = PCA(n_components=2, random_state=42)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

print(f"Erklärte Varianz durch 2 Komponenten: {pca.explained_variance_ratio_}")

# Scaler: [0, 2π]
scaler = MinMaxScaler(feature_range=(0, 2 * np.pi))
X_train_sc = scaler.fit_transform(X_train_pca)
X_test_sc = scaler.transform(X_test_pca)

print(f"Train: {X_train_sc.shape} | Test: {X_test_sc.shape}")
print(f"Klassenverteilung Test - malignant: {(y_test==0).sum()} benign: {(y_test==1).sum()}")
```

Erklärte Varianz durch 2 Komponenten: 99.93%
Train: (398, 2) | Test: (171, 2)
Klassenverteilung Test - malignant: 64 benign: 107

2. Quantum Kernel SVM

Feature Map: `zz_feature_map`, 2 Qubits, reps=2

```
In [3]: feature_map_qk = zz_feature_map(feature_dimension=2, reps=2)
kernel = FidelityQuantumKernel(feature_map=feature_map_qk)

start = time.time()
svc_q = SVC(kernel=kernel.evaluate)
svc_q.fit(X_train_sc, y_train)
train_time_qk = round(time.time() - start, 4)

start = time.time()
y_pred_qk = svc_q.predict(X_test_sc)
infer_time_qk = round(time.time() - start, 4)

acc_qk = accuracy_score(y_test, y_pred_qk)
f1_qk = f1_score(y_test, y_pred_qk, average="binary")
```

```

print(f"Accuracy: {acc_qk:.4f} | F1: {f1_qk:.4f}")
print(f"Training: {train_time_qk}s | Inferenz: {infer_time_qk}s")
print()
print(classification_report(y_test, y_pred_qk, target_names=bc.target_names))

```

Accuracy: 0.8772 | F1: 0.9041
 Training: 408.9585s | Inferenz: 352.8897s

	precision	recall	f1-score	support
malignant	0.86	0.80	0.83	64
benign	0.88	0.93	0.90	107
accuracy			0.88	171
macro avg	0.87	0.86	0.87	171
weighted avg	0.88	0.88	0.88	171

3. VQC (RealAmplitudes)

Feature Map: `zz_feature_map`, reps=1 | Ansatz: `real_amplitudes`, reps=2 |
 Optimizer: SPSA 150

```

In [4]: feature_map_vqc = zz_feature_map(feature_dimension=2, reps=1)
        ansatz = real_amplitudes(num_qubits=2, reps=2)

        vqc = VQC(
            feature_map=feature_map_vqc,
            ansatz=ansatz,
            optimizer=SPSA(maxiter=150),
            sampler=StatevectorSampler(),
        )

        print("Starte VQC Training...")
        start = time.time()
        vqc.fit(X_train_sc, y_train)
        train_time_vqc = round(time.time() - start, 4)

        start = time.time()
        y_pred_vqc = vqc.predict(X_test_sc)
        infer_time_vqc = round(time.time() - start, 4)

        acc_vqc = accuracy_score(y_test, y_pred_vqc)
        f1_vqc = f1_score(y_test, y_pred_vqc, average="binary")

        print(f"Accuracy: {acc_vqc:.4f} | F1: {f1_vqc:.4f}")
        print(f"Training: {train_time_vqc}s | Inferenz: {infer_time_vqc}s")
        print()
        print(classification_report(y_test, y_pred_vqc, target_names=bc.target_names))

```

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Starte VQC Training...
 Accuracy: 0.8713 | F1: 0.9027
 Training: 1004.3609s | Inferenz: 1.2259s

	precision	recall	f1-score	support
malignant	0.90	0.73	0.81	64
benign	0.86	0.95	0.90	107
accuracy			0.87	171
macro avg	0.88	0.84	0.86	171
weighted avg	0.87	0.87	0.87	171

4. Klassischer SVM (RBF-Kernel)

```

In [5]: start = time.time()
svc_rbf = SVC(kernel="rbf", random_state=42)
svc_rbf.fit(X_train_sc, y_train)
train_time_svm = round(time.time() - start, 4)

start = time.time()
y_pred_svm = svc_rbf.predict(X_test_sc)
infer_time_svm = round(time.time() - start, 4)

acc_svm = accuracy_score(y_test, y_pred_svm)
f1_svm = f1_score(y_test, y_pred_svm, average="binary")

print(f"Accuracy: {acc_svm:.4f} | F1: {f1_svm:.4f}")
print(f"Training: {train_time_svm}s | Inferenz: {infer_time_svm}s")
print()
print(classification_report(y_test, y_pred_svm, target_names=bc.target_names

```

Accuracy: 0.9181 | F1: 0.9381
 Training: 0.0076s | Inferenz: 0.0018s

	precision	recall	f1-score	support
malignant	0.98	0.80	0.88	64
benign	0.89	0.99	0.94	107
accuracy			0.92	171
macro avg	0.94	0.89	0.91	171
weighted avg	0.92	0.92	0.92	171

5. MLP (64, 32)

```

In [6]: start = time.time()
mlp = MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=2000, random_state
mlp.fit(X_train_sc, y_train)
train_time_mlp = round(time.time() - start, 4)

start = time.time()

```

```

y_pred_mlp = mlp.predict(X_test_sc)
infer_time_mlp = round(time.time() - start, 4)

acc_mlp = accuracy_score(y_test, y_pred_mlp)
f1_mlp = f1_score(y_test, y_pred_mlp, average="binary")

print(f"Accuracy: {acc_mlp:.4f} | F1: {f1_mlp:.4f}")
print(f"Training: {train_time_mlp}s | Inferenz: {infer_time_mlp}s")
print()
print(classification_report(y_test, y_pred_mlp, target_names=bc.target_names))

```

```

Accuracy: 0.9240 | F1: 0.9412
Training: 0.9143s | Inferenz: 0.001s

```

	precision	recall	f1-score	support
malignant	0.95	0.84	0.89	64
benign	0.91	0.97	0.94	107
accuracy			0.92	171
macro avg	0.93	0.91	0.92	171
weighted avg	0.93	0.92	0.92	171

6. Ergebnisse speichern

Label Breast Cancer (2 Klassen, 2 Features, fair) - konsistent mit Iris-Konvention.

```

In [7]: DATENSATZ = "Breast Cancer (2 Klassen, 2 Features, fair)"

save_result("Quantum Kernel SVM", DATENSATZ, "AerSimulator", acc_qk, f1_
save_result("VQC (RealAmplitudes)", DATENSATZ, "AerSimulator", acc_vqc, f1_
save_result("Klassischer SVM (RBF)", DATENSATZ, "lokal", acc_svm, f1_
save_result("MLP", DATENSATZ, "lokal", acc_mlp, f1_

print("Alle Ergebnisse gespeichert.")

```

	Modell		Datensatz		Backe
nd	Accuracy	F1	Trainingszeit_s	Inferenzzeit_s	Reps
Quantum Kernel SVM			Iris (3 Klassen, 2 Features, fair)	AerSimulat	
or	0.6222	0,616	28.2243	23,517 zz_feature_map	2.0
VQC (RealAmplitudes)			Iris (3 Klassen, 2 Features, fair)	AerSimulat	
or	0.5333	0.4222	261.5463	0.3237 zz_feature_map	1.0
Klassischer SVM (RBF)			Iris (3 Klassen, 2 Features, fair)		lok
al	0.9556	0.9556	0.0114	0.0012	NaN
MLP			Iris (3 Klassen, 2 Features, fair)		lok
al	0.9333	0.9327	0.3173	0.0012 (64, 32)	NaN
Quantum Kernel SVM Breast Cancer (2 Klassen, 2 Features, fair)			AerSimulat		
or	0.8772	0.9041	408.9585	352.8897 zz_feature_map	2.0

	Modell		Datensatz		Backe
nd	Accuracy	F1	Trainingszeit_s	Inferenzzeit_s	Reps
Quantum Kernel SVM			Iris (3 Klassen, 2 Features, fair)	AerSimulat	
or	0.6222	0,616	28.2243	23,517 zz_feature_map	2.0
VQC (RealAmplitudes)			Iris (3 Klassen, 2 Features, fair)	AerSimulat	
or	0.5333	0.4222	261.5463	0.3237 zz_feature_map	1.0
Klassischer SVM (RBF)			Iris (3 Klassen, 2 Features, fair)		lok
al	0.9556	0.9556	0.0114	0.0012	NaN
MLP			Iris (3 Klassen, 2 Features, fair)		lok
al	0.9333	0.9327	0.3173	0.0012 (64, 32)	NaN
Quantum Kernel SVM Breast Cancer (2 Klassen, 2 Features, fair)			AerSimulat		
or	0.8772	0.9041	408.9585	352.8897 zz_feature_map	2.0
VQC (RealAmplitudes) Breast Cancer (2 Klassen, 2 Features, fair)			AerSimulat		
or	0.8713	0.9027	1004.3609	1.2259 zz_feature_map	1.0

	Modell		Datensatz		Backe
nd	Accuracy	F1	Trainingszeit_s	Inferenzzeit_s	Reps
Quantum Kernel SVM			Iris (3 Klassen, 2 Features, fair)	AerSimulat	
or	0.6222	0,616	28.2243	23,517 zz_feature_map	2.0
VQC (RealAmplitudes)			Iris (3 Klassen, 2 Features, fair)	AerSimulat	
or	0.5333	0.4222	261.5463	0.3237 zz_feature_map	1.0
Klassischer SVM (RBF)			Iris (3 Klassen, 2 Features, fair)		lok
al	0.9556	0.9556	0.0114	0.0012	NaN
MLP			Iris (3 Klassen, 2 Features, fair)		lok
al	0.9333	0.9327	0.3173	0.0012 (64, 32)	NaN
Quantum Kernel SVM Breast Cancer (2 Klassen, 2 Features, fair)			AerSimulat		
or	0.8772	0.9041	408.9585	352.8897 zz_feature_map	2.0
VQC (RealAmplitudes) Breast Cancer (2 Klassen, 2 Features, fair)			AerSimulat		
or	0.8713	0.9027	1004.3609	1.2259 zz_feature_map	1.0
Klassischer SVM (RBF) Breast Cancer (2 Klassen, 2 Features, fair)					lok
al	0.9181	0.9381	0.0076	0.0018	NaN

	Modell		Datensatz		Backe
nd	Accuracy	F1	Trainingszeit_s	Inferenzzeit_s	Reps
Quantum Kernel SVM			Iris (3 Klassen, 2 Features, fair)	AerSimulat	
or	0.6222	0,616	28.2243	23,517 zz_feature_map	2.0
VQC (RealAmplitudes)			Iris (3 Klassen, 2 Features, fair)	AerSimulat	
or	0.5333	0.4222	261.5463	0.3237 zz_feature_map	1.0
Klassischer SVM (RBF)			Iris (3 Klassen, 2 Features, fair)		lok
al	0.9556	0.9556	0.0114	0.0012	NaN
MLP			Iris (3 Klassen, 2 Features, fair)		lok
al	0.9333	0.9327	0.3173	0.0012 (64, 32)	NaN
Quantum Kernel SVM Breast Cancer (2 Klassen, 2 Features, fair)			AerSimulat		
or	0.8772	0.9041	408.9585	352.8897 zz_feature_map	2.0
VQC (RealAmplitudes) Breast Cancer (2 Klassen, 2 Features, fair)			AerSimulat		
or	0.8713	0.9027	1004.3609	1.2259 zz_feature_map	1.0

```

Klassischer SVM (RBF) Breast Cancer (2 Klassen, 2 Features, fair)      lok
al      0.9181 0.9381              0.0076              0.0018      NaN      NaN
              MLP Breast Cancer (2 Klassen, 2 Features, fair)      lok
al      0.9240 0.9412              0.9143              0.001      (64, 32)      NaN
Alle Ergebnisse gespeichert.

```

7. Vergleichsübersicht

```

In [2]: df = pd.read_csv("Ergebnisse/ergebnisse.csv")
df_bc = df[df["Datensatz"].str.contains("Breast Cancer")]
print(df_bc[["Modell", "Backend", "Accuracy", "F1", "Trainingszeit_s", "Infe

```

	Modell	Backend	Accuracy	F1	Trai
ningszeit_s	Inferenzzeit_s				
	Quantum Kernel SVM	AerSimulator	0.8772	0.9041	
408.958500	352.8897				
	VQC (RealAmplitudes)	AerSimulator	0.8713	0.9027	
1004.360900	1.2259				
	Klassischer SVM (RBF)	lokal	0.9181	0.9381	
0.007600	0.0018				
	MLP	lokal	0.924	0.9412	
0.914300	0.001				
	Quantum Kernel SVM (Noise)	AerSimulator+Noise	0.8772	0.9041	
399.497200	345.1652				
	VQC (RealAmplitudes, Noise)	AerSimulator+Noise	0.8421	0.8831	
4600.403300	9.5284				
	VQC (RealAmplitudes, COBYLA)	AerSimulator	COBYLA	50	
0.836257	0.8727272727272727				
	VQC (RealAmplitudes, COBYLA, Noise)	AerSimulator+Noise	COBYLA	50	
0.824561	0.8660714285714286				